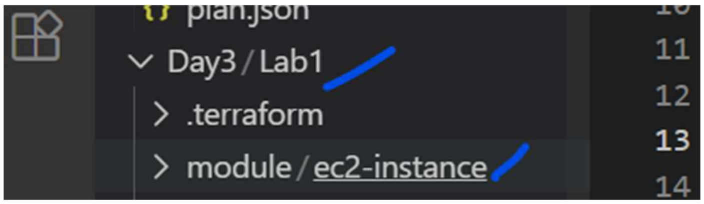
Steps

1. Create Day3 and Lab1
2. Create module folder inside Lab1
3. And create “ec2-instance” folder inside “module”



4. Create file “ec2.tf” inside the “ed2-instance” folder.

   Content of ec2.tf”

```
locals {
 tag_Name = "Viswa-${var.env}"
}

resource "aws_instance" "main" {
   ami = var.region_ami_map[var.region-1]
   instance_type = var.instance_type
   subnet_id = var.subnets[count.index]
   count = var.num_of_vm
   associate_public_ip_address = false

 tags = {
   "Name" = local.tag_Name
 }
}
```

5. Create “ec2_vars.tf” inside “ec2-instance folder

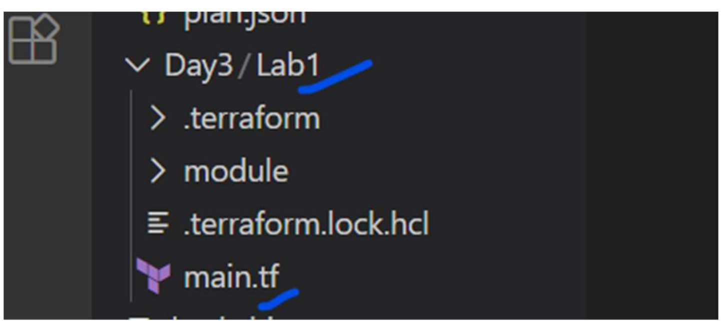
```
variable "region-1" {
default = "ap-south-1"
}
variable "env" {}
variable "num_of_vm" {
 type = number
    validation {
        condition = var.num_of_vm >= 1 && var.num_of_vm <= 10
        error_message = "Instance count must be between 1 and 10."
```

```
    }
}

variable "instance_type" {
  type = string
  validation {
    condition     = contains(["t3.nano", "t3.micro"], var.instance_type)
    error_message = "Only t3.nano or t3.micro are allowed."
  }
}

variable "subnets" {
    type = list
}

variable "region_ami_map" {
  type = map(string)
  default = {
    "ap-south-1" = "ami-0ced6a024bb18ff2e"
    "us-east-1" = "ami-12345"
  }
}
```

6. Create “ec2_output.tf”

```
output "vm_priv_pips" {
  value = [for vm in aws_instance.main : vm.private_ip ]
}
```

7. Finally create the main.tf for running the parents code.
   The file has to be in the “lab1” folder.



Main.tf content

provider "aws" {

region = "ap-south-1"

}

```
module "Vms_app1" {
  source = "./module/ec2-instance"
  region-1 = "ap-south-1"
  instance_type = "t3.micro"
  subnets = ["subnet-0a413d769fe98ba0b", "subnet-043f1f8151b681cca", "subnet-04f47a3252bdf06e6"]
  num_of_vm = 1
  env = "dev"
}

output "vm_private_ips01" {
  value = module.Vms_app1.vm_priv_pips
}
```

8. Running the terraform commands
   This has to be run inside the “Lab1” folder.

```
[ec2-user@ip-172-31-18-53 Lab1]$ pwd
/home/ec2-user/Day3/Lab1
[ec2-user@ip-172-31-18-53 Lab1]$ terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock fi
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan"
```

Terraform plan

```
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[ec2-user@ip-172-31-18-53 Lab1]$ terraform plan

Terraform used the selected providers to generate the following execution plan.
the following symbols:
  + create

Terraform will perform the following actions:

  # module.Vms_app1.aws_instance.main[0] will be created
  + resource "aws_instance" "main" {
      + ami                                  = "ami-0ced6a024bb18ff2e"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = false
      + availability_zone                    = (known after apply)
```